



OpenAPV: Open Collaborative Innovation in Professional Video Ecosystem

Minsoo Park
Samsung Electronics
Seoul, South Korea
mss.park@samsung.com

Youngkwon Lim
Samsung Electronics
Plano, TX, USA
young.l@samsung.com

Yangwoo Kim
Samsung Electronics
Seoul, South Korea
yearly.kim@samsung.com

Sam Richards
Academy Software Foundation
London UK
sam.richards@taurich.org

Min Woo Park
Samsung Electronics
Seoul, South Korea
m.w.park@samsung.com

Kwang Pyo Choi[†]
Samsung Electronics
Seoul, South Korea
kp5.choi@samsung.com

Abstract

Over the past few decades, significant resources have been invested in developing video codecs for the storage and delivery of video data. International standards development organizations, such as MPEG (Moving Picture Experts Group) and AOM (Alliance for Open Media), have fostered large-scale competitive collaboration among industry players. While open-source software developed alongside these video codec standards has accelerated verification of the standard and enabled rapid adoption, as it is used as a starting point for implementation and future research, this practice has not been fully applied to the ecosystem of mezzanine video codecs used in high-quality video capture and post-production. To address this gap, the industry's first open source project for collaborative innovation in professional video codecs, OpenAPV, has been established. The project aims to facilitate collaborative research and development of a royalty free, open source, and open standard video codec for professional use. This paper outlines the technical aspects of the video codec, together with the open source software implementation. The codec has been successfully tested across various platforms and shows excellent R-D (Rate-Distortion) performance and coding speed, even without hardware acceleration. Furthermore, it demonstrates robust quality resilience against multiple rounds of encoding and decoding cycles, which are common in video post-production. The open source project is hosted at <https://github.com/AcademySoftwareFoundation/openapv>.

CCS Concepts

- Computing methodologies~Computer graphics~Image compression
- Software and its engineering~Software creation

[†]Corresponding Author



This work is licensed under Creative Commons Attribution International 4.0.
MM '25, Dublin, Ireland

© 2025 Copyright is held by the owner/author(s).
ACM ISBN 979-8-4007-2035-2/2025/10.
<https://doi.org/10.1145/3746027.3756876>

and management~Collaboration in software development~Open source model

Keywords

Professional video; Advanced Professional Video (APV); OpenAPV; FFMpeg; Open Source; ASWF

ACM Reference format:

Minsoo Park, Youngkwon Lim, Yangwoo Kim, Sam Richards, Min Woo Park and Kwang Pyo Choi. 2025. OpenAPV: Open Collaborative Innovation in Professional Video Ecosystem. In *Proceedings of the 33rd ACM International Conference on Multimedia (MM '25)*, October 27-31, 2025, Dublin, Ireland. ACM, New York, NY, USA, 4 pages. <https://doi.org/10.1145/3746027.3756876>

1 Introduction

Increasing importance of social media highlighted the importance of quality of video captured by regular users with personal video recording devices. Capability and performance of hardware and software of personal video recording devices such as smartphones and digital cameras have been dramatically improved recently but there has been no noticeable enhancement in mezzanine video ecosystem. One of the main reasons is due to closed nature of the mezzanine video codec development and deployment. Large number of researchers were able to easily access to the specification and open source software of the conventional video codecs for media services such as HEVC, VVC or AV1 [1]-[3] to learn and test to add ideas to further improve them. However, mezzanine video codecs have not been developed through collaborative research and development. Widely used mezzanine video codecs are developed by a single company and offered through closed individual licensing. Therefore, no researches for improvements of the established mezzanine video codecs have been done by the third parties.

This paper introduces a mezzanine video codec open source software project, OpenAPV [4] to foster collaborative

Table 1: Major technologies of APV and the known references older than 20 years

Technology	Reference	Publication date
Tile partitioning	JPEG 2000 specification [6]	Aug. 2002
16x16 MB and 8x8 DCT transform	ITU-T H.261 specification [7]	Dec. 1990
Fixed point DCT	IEEE Transaction [8]	Jan. 1974
Uniform scalar quantization	ITU-T SG16 contribution [9]	Oct. 1998
Prediction in frequency domain	ITU-T T.81 specification [10]	Sept. 1992
Entropy coding	IEEE Transaction [11]	Mar. 1975

improvement of mezzanine video codec for wide industry use. The software is a conformant implementation of the Advanced Professional Video (APV), a royalty free open standard based mezzanine video codec [5]. The success of the above-mentioned conventional video codecs has proven that fruitful collaborative development of a video codec requires not only the open public specifications people can study but also the open source software conformant to the specification so that people can freely test new ideas to improve it.

To be used as an open platform of collaborative research and development, the codec has been designed to be royalty free and the software has been developed with the copy rights with no limits in use and redistribution.

Section 2 will introduce the design of APV and section 3 will describe the details of OpenAPV open source project. Experiment results in various aspects of OpenAPV will be summarized in section 4. Then section 5 concludes the paper.

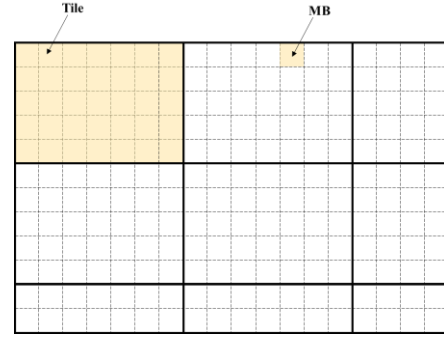
2 Design of APV

2.1 Design principle of APV

In addition to conventional requirements of mezzanine video codecs such as low complexity, minimum quality degradation during multiple rounds of encoding and decoding and so on, use of the technologies with the known references older than 20 years was one of the most important design principles of APV. By using such technologies, APV can be used without any major concern on royalties and risk of patent litigation so that it can be an open platform for collaborative research and development of a mezzanine video codec. Table 1 lists the major technologies used in APV and the example of the known references older than 20 years.

2.2 Architecture of APV

Overall APV codec architecture can be illustrated as follows. At APV encoder, original frame is first divided into one or more rectangular tiles [6] to support parallel processing and immersive video (e.g., 360-degree video, multi-view video, and so on). A tile is further divided into 16x16 pixel size macroblocks (MBs), a basic unit for compression. A MB is further divided into 8x8 pixel size transform blocks [7]. Each transform block is transformed using a fixed point DCT [8] and then transformed coefficients are

**Figure 1: Example of a frame partitioned into nine tiles**

quantized using uniform scalar quantizer [9]. After obtaining quantized coefficients, a prediction is applied in the frequency domain [10]. Finally, APV bitstream is generated through entropy coding that is specially designed to support high throughput and to fit very high bit rate range up to a few Gbps [11]-[14]. APV decoder processes APV bitstream in reverse order of the APV encoding process to produce decoded frame.

Figure 1 shows an example of a frame partitioned into nine tiles. The frame is divided into three tile columns and three tile rows, and each tile is divided into 35 MBs (7×5 MBs) except tiles at right and bottom boundaries of the frame.

2.3 Coding tools of APV

2.3.1 Transform and quantization. In APV, DCT transform is applied to original input signal to exploit spatial redundancy. Before performing the transform, the offset whose value is equal to the middle value of the possible pixel value range is subtracted from the original pixel value to create zero mean.

APV uses DCT transform with fixed-point operation to minimize mismatch between encoder and decoder.

The 2-D DCT transform with A can be applied to an input 8×8 block X to obtain the transform coefficients Y with the following equation.

$$Y = AXA^T$$

After the above transform is carried out, uniform linear scalar quantization is applied to each transform coefficient of Y to be able to optimize data rate and image quality. This is performed based on the given quantization parameter (QP).

2.3.2 Prediction. Prediction is performed in frequency domain unlike other conventional video codecs such as HEVC. The quantized DC coefficient is predicted from the one in the immediately preceding coded transform block in the MB within the same color component of a tile in raster scan order.

2.3.3 Entropy coding. Entropy coding of the quantized coefficients for each 8×8 transform blocks is performed with variable length coding. The quantized coefficients are scanned in

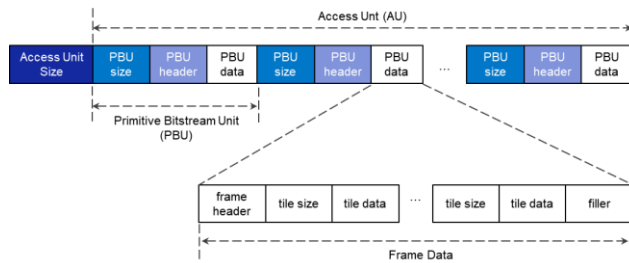


Figure 2: Overall structure of APV bitstream

zig-zag scanning pattern from the DC coefficient to the last AC coefficient in an 8×8 transform block.

The transform coefficient coding is designed to consist of a sequence of two syntax elements: level, which means the value of DC or non-zero AC coefficients, and zero-run, which means the number of zero coefficients from a level to next non-zero coefficient or from a level to the last AC coefficient in zig-zag scan order.

For coding of the syntax elements for level and run-zero, it is performed hybrid variable length coding scheme, which is combination of the simple prefix code which consists of three binary code words: “1”, “00”, “01”, and the Exponential Golomb code, which can be attached after the last prefix code “01”, which also includes the adjustment technique based on statistics of the rice parameter that determines the length of suffix code word.

The syntax elements for absolute value of level and run-zero are coded with the hybrid variable length code, and the sign of level is coded with one-bit flag.

2.4 Bitstream structure and metadata

The bitstream structure has been designed to enable efficient processing in each stage of capturing and post-processing. Figure 2 shows the overall structure of the APV bitstream. Each frame is independently coded to produce “Frame Data”. “Frame Data” consists of a “Frame Header”, one or more “Tile”(s), and optionally “Filler” at the end.

To collect more than one frame for single time instance for some applications such as multi-view or alpha channel, “Access Unit (AU)” is used. One or more “Frame Data” associated with same capture time or presentation time are converted into “Primitive Bitstream Unit (PBU)” by adding PBU size and PBU header, then concatenated to form an “Access Unit (AU).” Metadata is also added to AU as separate PBU. Individual metadata is formatted as Key-Length-Value structure and concatenated together. Externally defined metadata can be used without any modification by simply registering appropriate key value to identify them.

In APV, meta data for HDR10 and HDR10+ is defined to support HDR (High Dynamic Range) content. Additionally, user-defined metadata is permitted to support various types of videos.

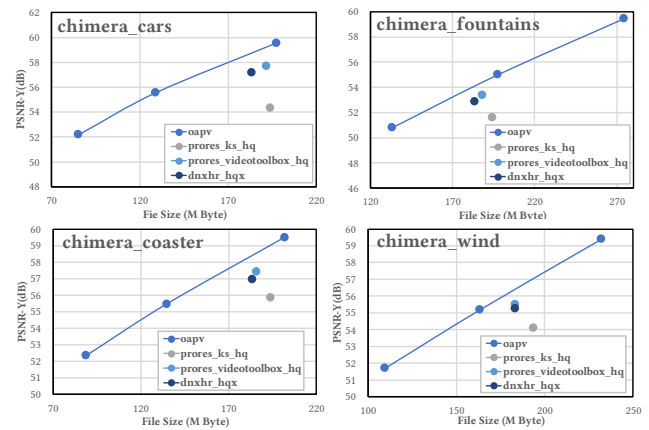


Figure 3: R-D performance comparison of OpenAPV against to FFMpeg implementation of ProRes and VC-3

3 OpenAPV project

The open source software project of APV, OpenAPV, has been established under the organization facilitating various open source software projects collaboratively developed and used by the content creation industry, the Academy Software Foundation (ASWF) [15]. The foundation has been jointly established by the Linux Foundation [16] and the Academy of Motion Picture Arts & Sciences (AMPAS) [17] in August 2018.

The OpenAPV project has been established under the ASWF as a sandbox project to develop a royalty free, open source, open standard video codec for professional video capturing and post-production. The 3-Clause BSD License [18] has been chosen as copyright of the project to allow almost unlimited free redistribution.

4 Experiment results

4.1 R-D performance

In order to measure the R-D (Rate-Distortion) performance of OpenAPV, the encoding guideline developed by ASWF for motion picture industry [19] was employed. As outlined in the guideline, well-known professional video codecs, ProRes [20] and SMPTE VC-3 [21], were selected as the benchmarks for comparison. FFMpeg [22] implementations of these codecs have been utilized in accordance with the guideline. For ProRes, a full software implementation, namely prores_ks, and the implementation utilizing hardware acceleration, namely prores_videotoolbox have been chosen. For SMPTE VC-3, DNxHD and DNxHR implementations have been selected.

Comparison was conducted for four Full-HD sequences with three different qualities by using several different QP values, 15, 20, 25 for OpenAPV. As FFMpeg implementations of ProRes and SMPTE VC-3 do not allow flexible modification of target quality, the predefined quality presets closest to the R-D curve of OpenAPV have been chosen. The high quality (HQ) preset for

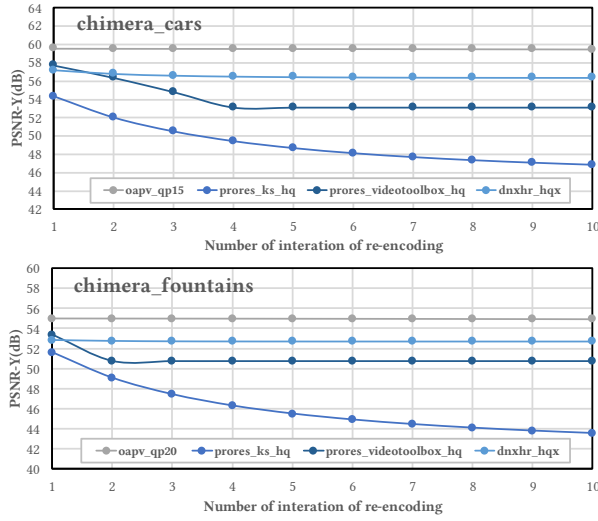


Figure 4: Comparison of quality degradation of OpenAPV and FFmpeg implementation of ProRes and VC-3 in multiple rounds of re-encoding of two sequences

ProRes, and high quality 12-bit (HQX) preset for DNxHR have been used.

Figure 3 shows that R-D curve of OpenAPV always stay on top of all of the points generated by other codecs and implementations, which demonstrates outstanding rate-distortion performance of OpenAPV over other codecs.

4.2 Resiliency to multiple rounds of re-encoding

Post-production of video content requires repeated decoding and re-encoding. Therefore, maintaining the visual quality over multiple rounds of re-encoding is an essential feature for mezzanine video codec.

In this section, the robust quality resiliency of OpenAPV against multiple rounds of encoding and decoding is presented. The experiment was conducted at a similar bitrate to the previous results. From the results shown in Figure 4, it can be seen that OpenAPV successfully maintained highest visual quality in PSNR even after the 10th iteration for both sequences tested. FFmpeg DNxHR has also shown very small quality degradation after 10th iteration for both sequences even though its overall PSNR was generally lower than OpenAPV. Both FFmpeg prores_ks and FFmpeg prores_videotoolbox showed large quality drop during the first few iterations for both sequences, while the latter was able to maintain the degraded low quality during the rest of the iterations.

4.3 Real-time encoding/decoding performance

In this section, the encoding and decoding time performance is examined for OpenAPV. The implementation incorporates Single Instruction Multiple Data (SIMD) optimizations, and multi-threading using tile-level parallelism. Additionally, for entropy coding in the encoder, a predefined lookup table is utilized. The

Table 2: Multi-thread encoding and decoding execution time of Full-HD sequences

	Encoding speed					Decoding speed				
# of threads	1	2	4	8	16	1	2	4	8	16
frames/second	84	162	288	495	651	56	162	288	348	390

execution time was measured on a machine with Intel Ultra7 265KF CPU and 64 GB RAM. Table 2 show that real-time encoding and decoding of Full-HD 60 fps is easily achievable without any hardware acceleration.

5 Conclusion

This paper introduces open source software project, OpenAPV, to foster collaborative improvement of mezzanine video codec for wide industry use hosted by the ASWF. The experimental results have proven that OpenAPV already shows exceptional R-D performance and fulfills important requirements of mezzanine video ecosystem. Outstanding performance of the mezzanine codec utilizing the technologies older than 20 years demonstrated the strong potential of continuous improvements of OpenAPV as more and more advanced technologies can be added to the project when they become more than 20 years old. Active involvements of the motion picture industry through the ASWF is expected to guide fine tuning of OpenAPV to the right direction and the sweet spots.

References

- [1] Rec. ITU-T H.265 | ISO/IEC 23008-2, 2013, High efficiency video coding.
- [2] Rec. ITU-T H.266 | ISO/IEC 23090-3, 2021, Versatile video coding.
- [3] AV1-Spec, 2019, AV1 Bitstream Decoding Process Specification, <https://aomediacodec.github.io/av1-spec/av1-spec.pdf>.
- [4] OpenAPV, <https://github.com/AcademySoftwareFoundation/openapv>
- [5] IETF Internet-Draft, 2024, Advanced Professional Video, <https://datatracker.ietf.org/doc/draft-lim-apv>
- [6] Rec. ITU-T T.800 | ISO/IEC 15444-1, 2002, JPEG 2000 image coding system.
- [7] Rec. ITU-T H.261, 1998, Video codec for audiovisual services at p x 64 kbit/s.
- [8] N. Ahmed, T. Natarajan and K. R. Rao, 1974, Discrete Cosine Transform, in IEEE Transactions on Computers, vol. C-23, no. 1, pp. 90-93
- [9] G. Bjøntegaard, 1998, Response to Call for Proposals for H.26L, Q15-F-11, ITU-T Study Group 16, Q.16.
- [10] Rec. ITU-T T.81, 1992, Digital compression and coding of continuous-tone still images.
- [11] P. Elias, 1975, Universal codeword sets and representations of the integers, in IEEE Transactions on Information Theory, vol. 21, no. 2, pp. 194-203.
- [12] D. Marpe, 2001, Improved CABAC, VCEG-O18, ITU-T Study Group 16, Q.6.
- [13] J. Wen, P. Meshkat and J. Villasenor, 1996, Structured trees for lossless coding of quantized wavelet coefficients, Conference Record of The Thirtieth Asilomar Conference on Signals, Systems and Computers, Pacific Grove, CA, USA, pp. 931-937 vol.2.
- [14] N. Memon, 1998, Adaptive coding of DCT coefficients by Golomb-Rice codes, Proceedings 1998 International Conference on Image Processing, Chicago, IL, USA, pp. 516-520 vol.1.
- [15] Academy Software Foundation, <https://www.aswf.io>
- [16] Linux Foundation, <https://www.linuxfoundation.org>
- [17] Academy of Motion Picture Arts & Sciences, <https://www.oscars.org>
- [18] The 3-Clause BSD License, <https://opensource.org/licenses/bsd-3-clause>
- [19] ASWF Encoding Guidelines, <https://github.com/AcademySoftwareFoundation/EncodingGuidelines/>
- [20] SMPTE RDD 36, 2015, Apple ProRes Bitstream Syntax and Decoding Process.
- [21] SMPTE ST 2019-1, 2016, VC-3 Picture Compression and Data Stream Format.
- [22] FFmpeg, <https://ffmpeg.org/>